



리눅스 시스템 프로그래밍

3장 파일시스템

목차

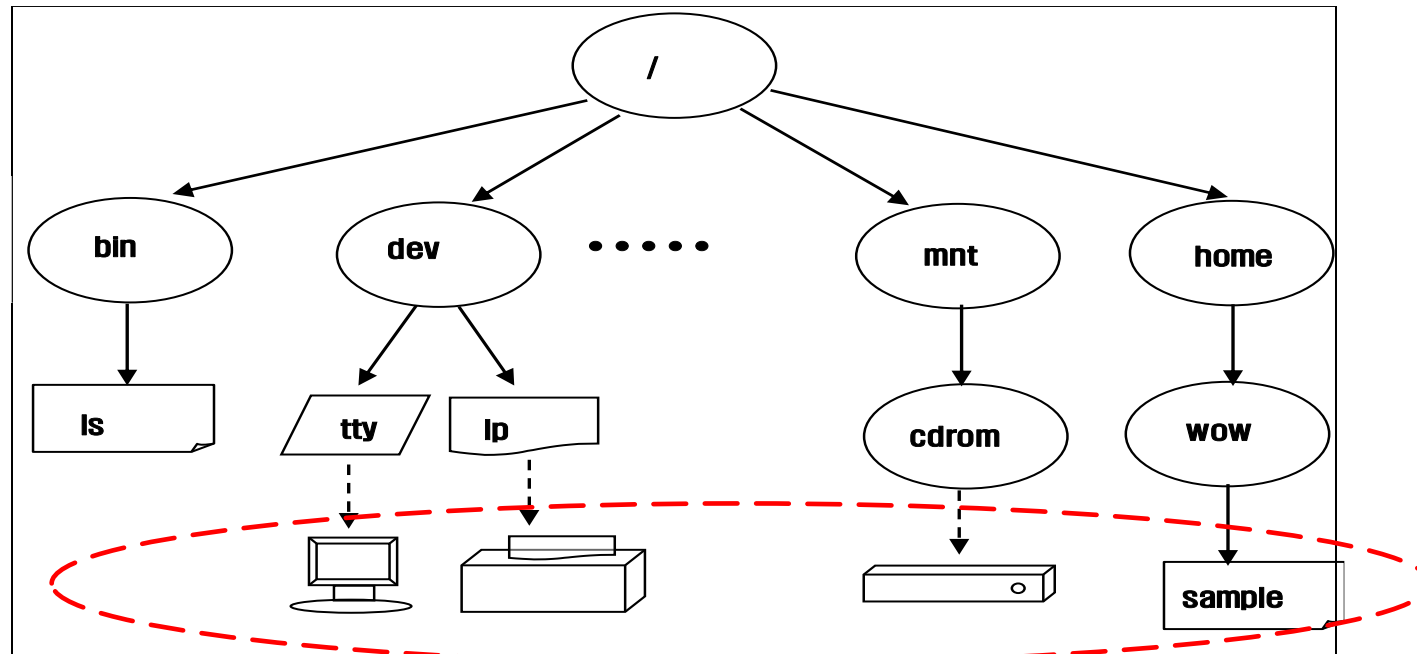
1. 파일시스템 구조
2. 파일 정보
3. 파일 입출력

파일시스템의 구조

1. 파일의 종류와 구조
2. 파일시스템 구조

파일의 종류와 구조

- 리눅스에서 각종 파일 다루는 함수를 알아보기에 앞서 좀더 효과적인 파일 처리를 위해 리눅스 파일 처리에 관한 기본 적인 사항들을 이해할 필요가 있다. 따라서 우선 리눅스에서 지원되는 파일의 종류와 시스템 구조를 살펴보기로 한다.
- 리눅스에는 모든 파일이 하나의 트리 구조 형태로 구성된다. 가장 위쪽에는 루트 라고 불리는 / 디렉토리가 존재하고 그 이하에 계층적으로 파일들이 구성된다. 다음은 리눅스 파일 트리의 구성을 간략하게 표현한 그림이다.

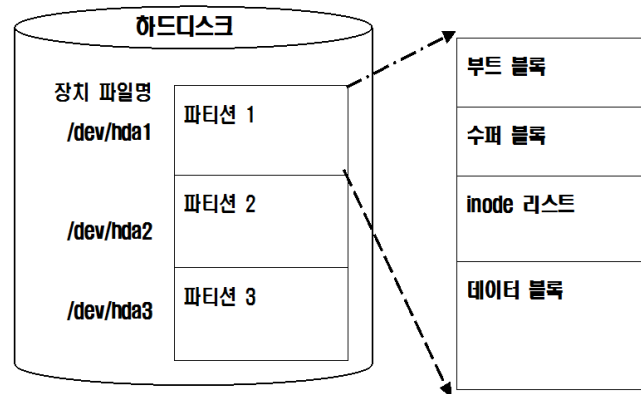


파일의 종류와 구조

- 앞의 그림에서 볼 수 있듯이 리눅스에는 다양한 유형의 파일들이 존재한다. 리눅스에서 지원되는 파일의 종류는 다음과 같다.
 - 일반 파일- 문서 파일, 실행 파일
 - 디렉토리
 - 장치 파일- 블록 장치 파일, 문자 장치 파일
 - 파이프 파일
 - 심볼릭 링크 파일
 - 소켓 파일
- 여기에서 디렉토리 및 블록 장치를 제외하고는 모든 파일을 동일하게 제어할 수 있다. 다시 말해 일반 파일의 입출력 함수를 다른 유형의 파일에도 적용할 수 있다는 것이다. 프로그램을 실행할 때, 즉 프로세스가 생성되었을 때 프로세스의 구조 중 파일 디스크립터 테이블을 통해 파일을 제어하므로 프로세스안에서는 동일한 시스템 콜을 통해 파일들을 이용할 수 있도록 되어있다. 이 점은 리눅스에서 프로그램 개발을 쉽게 할 수 있게 해주는 매력적인 점이다.

파일시스템의 구조

- 앞서 언급했던 리눅스의 파일 트리는 하드디스크나, CDROM, 플로피 디스크, PCMCIA 카드등과 같은 보조 기억 장치에 저장된다. 이 기억 장치에서는 파일시스템의 형식으로 파일 트리가 저장되고 관리되어진다. 리눅스에서는 파일들을 체계적으로 관리하기 위하여 ext2, ext3, ext4 파일시스템을 사용하고 있다. 이 파일시스템의 구조를 간략하게 표현해보면 다음과 같다.



- 각 파티션은 파일을 체계적으로 부트 블록, 수퍼 블록, inode 리스트, 데이터 블록 등으로 구성된다. 각 블록의 구성은 다음과 같다.

파일시스템의 구조

Boot Block	Super Block	Inode Block	Data Block
부트 프로그램이 포함	파일시스템 관리에 필요한 정보 - 파일시스템 크기 및 상태 - 현재 파일 개수 - 현재 사용중인 데이터 블록 수 - free 데이터 블록 리스트 - free inode 리스트	각 파일별로 관리에 필요한 정보 - 파일 종류 : 일반 파일, 디렉토리, 장치 파일 등 - 파일 사용 권한 : owner, group, other 에 대한 읽기, 쓰기, 실행 권한 - 링크 수 - 파일 소유자 - UID/GID - 최종 액세스 시각 - 최종 수정 시각 - 최종 변경 시각 - 파일 크기 - 데이터 블록을 가리키는 포인터	디렉토리 및 파일의 내용

- 리눅스에서는 윈도우 운영체제와 마찬가지로 하나의 디스크를 파티션으로 구분하여 각 영역을 별도로 관리하는 기능이 제공된다. 윈도우와 다른 점은 각 파티션 별로 별도의 파일 트리를 구성하는 것이 아니라 하나의 루트 디렉토리 아래에 모든 파티션의 파일들이 구성된다는, 즉 하나의 파일 트리로 구성된다는 점이다.

파일 정보

- 1.파일의 정보 확인
- 2.파일의 정보 변경

파일의 정보 확인

- 앞서 학습한 리눅스 파일시스템 구조 중 inode 리스트에는 각 파일에 대한 정보가 보관되어 있었다. 이 inode 리스트에서 특정 파일에 대한 정보를 조회하기 위한 함수로는 `stat()`, `lstat()`, `fstat()` 등이 있다. 이 함수들은 파일에 대한 정보, 즉 inode 리스트의 정보를 확인할 수 있는 시스템 콜로 그 원형은 다음과 같다.

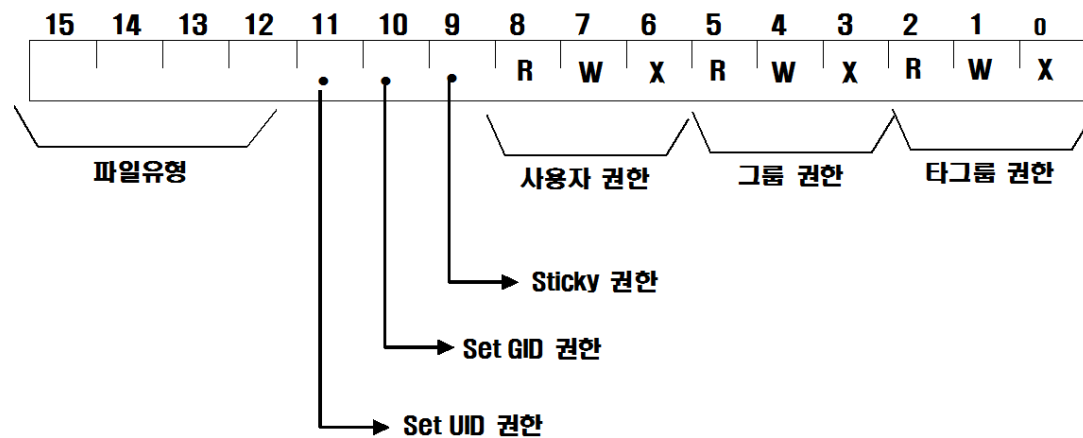
```
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
```

```
int stat(const char *path, struct stat *buf);
int lstat(const char *path, struct stat *buf);
int fstat(int fd, struct stat *buf);
```

```
struct stat {
    dev_t  st_dev;      // 디바이스 파일의 major/minor 번호
    ino_t   st_ino;      // inode 번호
    mode_t  st_mode;     // 파일의 종류와 파일의 권한
    nlink_t st_nlink;    // 링크 수
    uid_t   st_uid;     // 사용자의 UID
    gid_t   st_gid;     // 사용자의 GID
    dev_t   st_rdev;     // raw 디바이스 파일의 번호
    off_t   st_size;     // 파일 크기(byte 단위)
    blksize_t st_blksize; // 파일시스템의 블록 크기
    blkcnt_t st_blocks;  // 할당된 블록 수
    time_t  st_atime;    // 최종 액세스 시각
    time_t  st_mtime;    // 최종 수정 시각
    time_t  st_ctime;    // 최종 inode 정보 변경 시각
};
```

파일의 정보 확인

- 이 세 함수는 모두 파일에 관련된 정보를 inode 리스트로부터 읽어오는 함수로 두 번째 인수로 전달하는 변수에 읽어온다. `stat()`, `lstat()` 함수는 파일의 이름을 인수로 전달하는 반면 `fstat()` 함수에서는 파일 디스크립터를 인수로 전달한다. 따라서 `fstat()` 함수를 사용하려면 미리 `open()` 함수를 이용하여 파일 디스크립터를 얻어 와야 한다. `stat()` 함수와 `lstat()` 함수의 차이점은 심볼릭 링크된 파일 접근 시 `lstat()` 함수는 링크 파일 자신에 대한 정보를 읽어오는 반면은 `stat()` 함수에서는 링크된 원본 파일에 대한 정보를 읽어온다. 읽어 온 파일의 정보 중 `st_mode` 멤버는 파일의 유형과 사용 권한을 나타내는 것으로 다음과 같은 비트 디스크립션을 갖고 있다.



〈파일 정보 중 `st_mode` 멤버의 비트디스크립션〉

파일의 정보 확인

- 이 st_mode 멤버에 설정된 값들을 쉽게 분석할 수 있도록 sys/stat.h 헤더 파일에 매크로가 정의 되어 있다. 자주 사용되는 표현만 살펴보면 다음과 같다.

파일 유형과 관련된 매크로 함수	
S_ISDIR(st_mod)	디렉토리인 경우 참
S_ISREG(st_mod)	일반 파일인 경우 참
S_ISBLK(st_mode)	블록 장치 파일인 경우 참
S_ISCHR(st_mode)	문자 장치인 경우 참
S_ISFIFO(st_mode)	FIFO 파일인 경우 참
S_ISSOCK(st_mode)	Socket 파일인 경우 참
S_ISLNK(st_mode)	심볼릭 링크 파일인 경우 참
파일 권한에 관련된 마스크	
S_IRWXU	사용자의 읽기/쓰기/실행 권한
S_IRWXG	같은 그룹 사용자의 읽기/쓰기/실행 권한
S_IRWXO	다른 그룹 사용자의 읽기/쓰기/실행 권한
S_ISUID	Set UID 권한
S_ISGID	Set GID 권한
S_IRUSR	사용자의 읽기 권한
S_IWUSR	사용자의 쓰기 권한
S_IXUSR	사용자의 실행 권한



파일의 유형과 권한을 확인하는 프로그램을 작성해봅시다.

단, 파일명은 다음과 같이 전달 됩니다.

\$./ex01 파일명

파일의 정보 변경

■ 파일 권한 변경

우선 파일의 권한을 변경하는 함수는 `chmod()` 로 원형은 다음과 같다.

```
#include <sys/stat.h>

int chmod(const char *path, mode_t mode);
```

여기에서 첫 번째 인수에는 파일명을 두 번째 인수에는 파일의 권한을 8진수로 직접 기술하거나 `stat()` 함수에서 사용하는 사용 권한에 관련된 마스크 값을 비트 OR(I) 를 이용하여 설정한다.

■ 파일 소유자 변경

다음 `chown()` 함수를 이용하면 파일을 만든 생성자도 변경할 수 있다.

```
#include <unistd.h>

int chown(const char *path, uid_t owner, gid_t group);
```

여기에서 `path`는 파일명을, `owner`에는 변경할 UID를, `group` 에는 변경할 GID를 인수로 전달한다. 이 함수는 사용자 변경 권한 있는 사용자만 수행할 수 있다.

파일의 정보 변경

■ 파일 링크

파일 링크를 통해 다른 이름으로 공유할 수 있게 해주는 함수다.

```
#include <unistd.h>
```

```
int link(const char *path1, const char *path2);  
int symlink(const char *path1, const char *path2);
```

link() 함수는 하드링크 방식으로 링크하는 함수로 파일의 inode를 공유하므로 같은 파일을 가리킨다. 이에 비해 symlink() 함수는 원본 파일의 이름을 통해 파일을 공유하는 함수로 링크 방식은 다르나 동일한 파일을 가리킨다.

■ 파일 삭제

리눅스에서 파일이나 디렉토리를 삭제하려면 unlink() 함수를 이용한다.

```
#include <unistd.h>
```

```
int unlink(const char *path);
```

다음 프로그램을 입력한 후 실행하면서 파일 정보 변경 함수를 이해해 보시다.

소스코드 (ex02.c)

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>

int main(void)
{
    struct stat fstatbuf;

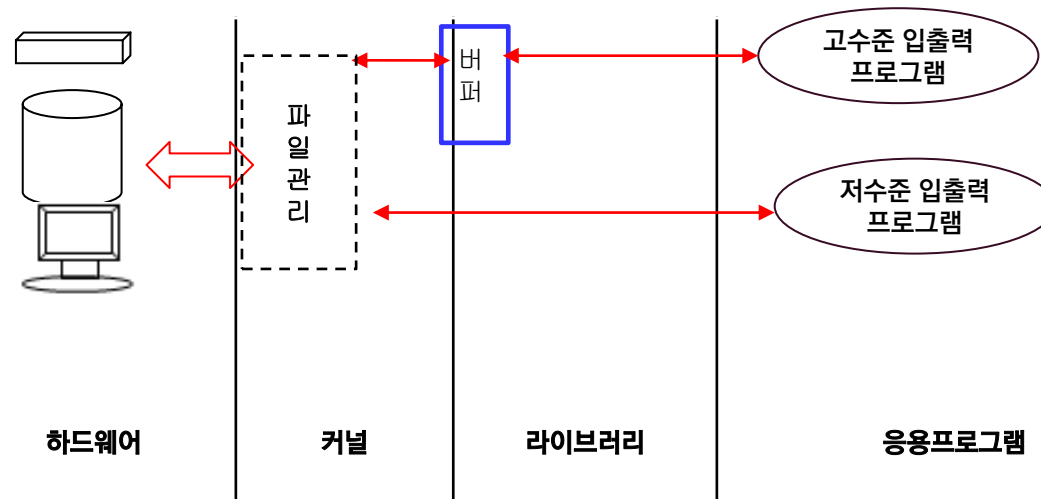
    system("touch data; ls -l data");
    stat("data", &fstatbuf);
    printf("\nchmod()\n");
    chmod("data", fstatbuf.st_mode|S_IXUSR|S_IXGRP);
    system("ls -l data");
    link("data", "data1");
    symlink("data", "data2");
    printf("\nlink() & symlink()\n");
    system("ls -li data data1 data2");
    return 0;
}
```

파일 입출력

1. 저 수준 vs 고수준의 입출력 방식
2. 저 수준의 입출력 함수

저 수준 VS 고수준의 입출력 방식

- 파일의 입출력 방식에는 다음과 같은 두 가지 방식이 있다.
 - 저수준 입출력
 - 고수준 입출력
- 두 입출력 방식의 차이점은 다음 그림과 같이 표현할 수 있다.



[그림] 고수준과 저수준 파일 입출력 방식

저 수준 VS 고수준의 입출력 방식

■ 고 수준 vs 저수준 입출력 방식 비교

기준	고수준	저수준
파일지시자	파일 포인터 (FILE *)	파일 디스크립터(int)
버퍼사용 유무	사용	사용 안함
세밀한 제어	어려움	쉬움
입출력 단위	스트림	바이트
표준화 여부	ANSI 표준	비 표준

■ 고수준 입출력 함수

```
FILE *fopen(char *filename, char *filemode);  
int size_t fread(void *ptr, size_t size, size_t n, FILE *fp);  
size_t fwrite(const void *ptr, size_t size, size_t n, FILE *fp);  
int fprintf(fp, char *format, variable list);  
int fscanf(FILE *fp, char *format,variable list);  
int feof(FILE *fp);  
FILE *fclose(FILE* fp);
```

저 수준 파일 입출력 함수

- 리눅스에서 파일의 내용을 읽거나 저장하려면 우선 `open()` 함수부터 수행해야 한다. 이 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open(const char *path, int oflags, (mode_t mode));
int close(int fd);
```

- `open()` 함수는 일반 파일이나 장치 파일 등에 대한 접근 경로를 만들어주는 함수로 파일 명과 오픈 플래그를 인수로 전달하면 접근 경로로 파일 디스크립터를 반환한다. `close()` 함수는 파일의 접근 경로를 해제할 때 이용하는 함수로 `open()` 함수의 리턴 값인 파일 디스크립터를 인수로 전달한다.
- 여기에서 파일 디스크립터란 앞서 살펴보았듯이 프로세스에서 사용하는 모든 파일과의 인터페이스로, 파일의 입출력은 모두 이 파일 디스크립터를 이용하여 수행한다. 이 중 0,1,2는 표준 입력, 표준 출력, 표준 오류 장치로 예약이 되어있다. 이 값은 0 이상이어야 하며, -1이 반환되었다면 `open()` 함수가 실패하였음을 의미한다.

저 수준 파일 입출력 함수

- `open()` 함수의 두 번째 인수로 사용되는 오픈 플래그 중 많이 사용되는 플래그는 다음과 같다

플래그	의미
O_RDONLY	읽기 전용으로 열기
O_WRONLY	쓰기 전용으로 열기
O_RDWR	읽기/쓰기 모두 가능하게 열기
O_CREAT	파일이 존재하지않는 경우 생성 세번째 인수에 사용 권한에 대한 mode 설정
O_TRUNC	파일이 존재하는 경우 내용을 없앴
O_APPEND	파일 뒤로 추가하기, 쓰기 모드와 함께 사용
O_NONBLOCK	non-block 모드로 열기
O_SYNC	파일의 내용이 변경될 때마다 저장장치에 즉시 저장하기

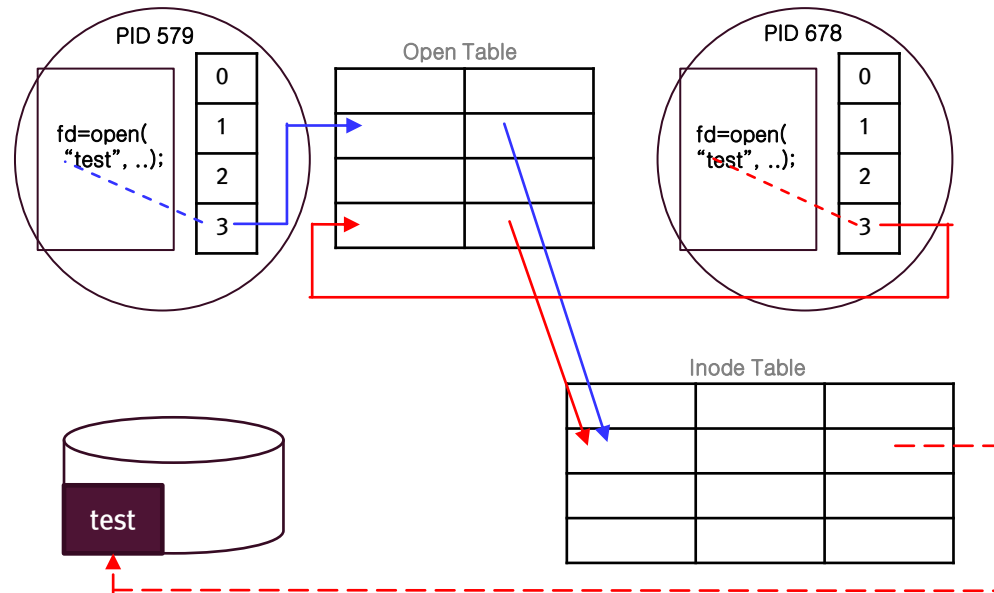
- `open()` 함수를 호출하게 되면 프로세스와 커널 사이에 어떤 관계가 이루어지는지 알아보는 것은 프로세스에서 파일 처리 방법을 이해하는데 많은 도움이 된다. 다음과 같이 2개의 프로세스에서 각각 `open()` 함수를 호출했다고 하자.

저 수준 파일 입출력 함수

〈PID 579이 두 프로세스와 커널과의 관계를 표현해 보면 다음 그림과 같다.

〈PID 579〉

```
fd=open( "test" , O_RDWR);
```



open 함수와 리눅스 커널

〈PID 678〉

```
fd=open( "test" , O_RDONLY);
```

open() 과 close() 함수를 이용하여 다음과 같은 예제 프로그램을 작성해보자.

소스코드 (ex03.c)

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>

int main(void)
{
    int fd;

    if((fd=open("/etc/hosts", O_RDONLY))==-1){
        perror("open1");
        fprintf(stderr, "File Read Fail.....\n");
    } else {
        printf("File Read Success!! fd= %d\n", fd);
        close(fd);
    }
    if((fd=open("myhosts", O_WRONLY|O_CREAT|O_TRUNC, 0666))==-1){
        perror("open2");
        fprintf(stderr, "File Create Fail.....\n");
    } else {
        printf("File Creat Success!! fd= %d\n", fd);
        close(fd);
    }
    return 0;
}
```

저 수준 파일 입출력 함수

■ read(), write() 함수

- open() 함수를 이용하여 파일을 열기를 성공하면 파일의 내용을 읽어오거나 저장할 때 read(), write() 함수를 사용할 수 있다. 원형은 다음과 같다.

```
#include <unistd.h>
```

```
size_t read(int fd, const void *buf, size_t nbytes);  
size_t write(int fd, const void *buf, size_t nbytes);
```

- read() 함수는 fd 로 연결된 파일로부터 nbytes 만큼의 데이터를 읽어 buf에 저장하는 함수로 실제로 읽어드린 크기를 반환한다. 만약 더 이상 읽을 것이 없을 때에는 0을, 함수 호출 시 문제가 발생한 경우에는 -1을 반환한다. write() 함수는 buf에 있는 내용을 nbytes 만큼 fd에 연결된 파일에 저장하는 함수로 실제로 저장된 바이트 수를 반환하며 아무것도 저장하지 않았을 때에는 0을, 저장 시 오류가 발생하였다면 -1을 반환한다.



ex03.c 소스코드를 ex04.c 로 복사하고 read(), write() 함수를 추가하여 파일의 내용을 복사해 봅시다.

저 수준 파일 입출력 함수

- lseek() 함수

read(), write() 함수는 파일을 순차적으로 접근하는 함수다. 함수 호출 때마다 내부에 있는 포인터를 이용하여 파일의 다음 부분을 순서대로 가리키고 있다. 때로 파일로 순차적인 접근이 아니라 임의의 접근이 필요한 경우도 있다. 이럴 때 사용할 수 있는 함수가 lseek() 다. 원형은 다음과 같다.

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

- 여기에서 fd는 파일디스크립터를, offset은 whence를 기준으로 한 오프셋을, whence는 기준 위치를 의미한다. whence에 설정할 수 있는 값은 다음과 같다

whence	값	의미
SEEK_SET	0	파일의 시작 위치
SEEK_CUR	1	파일 포인터의 현재 위치
SEEK_END	2	파일의 마지막 위치

다음과 같은 ex05.c 를 작성하고 lseek() 함수의 동작을 이해하여 봅시다.

소스코드 (ex05.c)	
<pre>..... int main(void) { int fd; char buf[256]; if((fd=open("myhosts",O_RDONLY))==-1) { perror("open"); exit(1); } lseek(fd, 10L, SEEK_SET); read(fd, buf, 10); buf[10]='\0'; printf("SEEK_SET : %s\n", buf); printf("-----\n\n");</pre>	<pre> lseek(fd, 10L, SEEK_CUR); read(fd, buf, 10); buf[10]='\0'; printf("SEEK_CUR : %s\n", buf); printf("-----\n\n"); lseek(fd, -10L, SEEK_END); read(fd, buf, 10); buf[10]='\0'; printf("SEEK_END : %s\n", buf); close(fd); return 0; }</pre>

〈실습〉

- 다음과 같이 파일 이름을 입력 받아 파일의 내용을 출력하는 프로그램을 작성해봅시다.

〈실행〉

`./exercice` *file_name*



리눅스 시스템 프로그래밍

4장 특수 파일 제어



목차

1. 터미널 장치 파일로의 접근
2. 파일 중첩
3. 디렉토리 제어

터미널 장치로의 접근

- 1.터미널 장치로의 입출력
- 2.터미널 장치의 속성 제어

터미널 장치로의 입출력

- 리눅스에서는 각종 장치들도 일반 파일을 제어하는 방법과 동일하게 파일 개념으로 제어한다. 따라서 `open()`, `read()`, `write()`, `close()` 함수를 이용하여 파일 입출력, 즉 장치로부터 데이터를 송수신 할 수 있다. 파일 입출력 함수에 대해서는 이미 살펴본 내용이므로 `ex01.c` 를 바로 살펴보기로 하자

소스코드 (ex01.c)

```
.....  
  
int main(int argc, char *argv[])  
{  
    char buffer[1024];  
    int fd, n;  
  
    if(argc!=2) {  
        fprintf(stderr, "Usage : ex01 device_filename\n");  
        exit(1);  
    }  
  
    if((fd=open(argv[1],O_RDWR))==-1){  
        perror("open");  
        exit(1);  
    }  
    strcpy(buffer, "Hello, Terminal\n");  
    n=strlen(buffer);  
    write(fd,buffer,n);  
    close(fd);  
}
```

터미널 장치의 속성 제어

- 리눅스에서 장치 드라이버를 개발하는 경우에는 파일 입.출력하는 함수 이외에 ioctl() 함수에 대응되는 장치 제어 함수를 만들고 각 속성을 제어하기 위한 명령들을 구현해야 한다. 사용자들은 이 ioctl() 함수를 통해 약속된 명령을 전달함으로써 장치의 속성들을 제어할 수 있게 된다.
- ioctl() 함수의 원형은 다음과 같다.

```
#include <termio.h>
```

```
int ioctl(int fd, int request, struct termios *tbuf);
```

- 여기에서 fd는 파일 디스크립터를, request는 장치 드라이버에게 처리를 부탁하는 요구 명령을, tbuf에는 장치에 관련된 정보를 받아오거나 전달할 때 사용할 구조체 변수의 주소를 넣어준다. 이때 구조체 자료형은 장치 개발자가 정의한 내용에 맞춰 기술한다.
- 터미널 장치의 경우 struct termios 구조체를 기술한다. 이 함수의 수행이 정상적이라면 0 이상의 정수를 반환하고, 오류가 발생하면 -1을 반환한다. 장치의 속성을 제어하려면 우선 장치 파일을 open() 해야 한다. 여기에서는 손쉽게 확인할 수 있는 터미널 장치의 속성을 제어해보자.

터미널 장치의 속성 제어

- 터미널 장치는 표준 입출력 장치로 이용되므로 0번 디스크립터를 이용하여 사용하기로 한다. 터미널 장치와 관련된 request에는 어떤 것들이 있는지 알아보자.

Request 종류	의미
TCGETA	터미널 속성정보를 termio 구조체 변수 tbuf에 읽어온다.
TCSETA	tbuf에 지정된 값을 터미널 장치에 설정한다.
TCSETAW	새로운 속성정보를 설정하기 전에 출력이 완전히 끝나기를 기다린다.
TCSETAF	입출력 버퍼에 있는 데이터를 모두 처리한 후 새로운 속성정보로 설정한다.

- 터미널 장치에 전달할 수 있는 request는 이외에도 더 많은 종류가 있다. 더 자세한 사항은 `termios.h` 헤더 파일을 참조하도록 한다. 위와 같은 request를 통해 터미널 제어 정보를 읽어보거나 변경할 수 있는데 이때 사용되는 `termios` 구조체 자료형은 다음과 같다.

```
#define NCC          32
struct termios {
    unsigned short c_iflag;           // 입력 모드 플래그
    unsigned short c_oflag;           // 출력 모드 플래그
    unsigned short c_cflag;           // 제어모드 플래그
    unsigned short c_lflag;           // 로컬 모드 플래그
    unsigned char c_cc[NCC];          // 제어문자
};
```

터미널 장치의 속성 제어

- 구조체 각 멤버들은 각각 터미널을 제어하는 속성 정보와 밀접한 연관이 있다.
- 각 멤버에 표현할 수 있는 매크로 중 대표적인 것들만 살펴보면 다음과 같다.

입력모드에 유효한 매크로 (c_iflag)	
ISTRIP	8 비트 문자를 7 비트로 스트립한다.
INLCR	개행 문자(LF) 을 복귀 문자(CR)로 변환한다.
ICRNL	복귀 문자(CR)를 개행 문자(LF)로 변환한다.
IXON	흐름을 제어할 수 있다.
출력 모드에 유효한 매크로 (c_oflag)	
OPOST	출력 모드 변경을 가능하게한다.
OCRNL	복귀 문자(CR)를 개행 문자(LF)로 변경하여 출력한다.
ONLCR	개행 문자(LF)를 복귀 문자(CR)로 변경하여 출력한다.

제어 모드에 유효한 매크로 (c_cflag)	
CBAUD	전송속도를 지정한다.(baud rate)
PARENB	패리티를 유효하게 만든다.
CSIZE	데이터의 길이를 지정한다. (5 비트 ~ 8 비트)
CSTOPB	정지 비트를 2로 설정한다. (아니면 1)
로컬 모드에서 유효한 매크로 (c_lflag)	
ICANON	입력 시 문자들을 제어한다.(backspace, DEL 등 처리)
ECHO	입력 문자를 다시 에코한다.
ECHOE	Erase 문자를 BS-SP-BS 로 에코한다.
ECHONL	개행 문자를 에코한다.

터미널 장치의 속성 제어

- 마지막 멤버인 `c_cc` 배열에는 다음과 같은 각종 제어 문자가 포함되어있다

배열 원소	제어 문자	일반 설정
<code>c_cc[VINTR]</code>	INTR(인터럽트)	Control C
<code>c_cc[VQUIT]</code>	QUIT(중지)	Control \
<code>c_cc[VERASE]</code>	ERASE(지우기)	DEL
<code>c_cc[VKILL]</code>	KILL(없애기)	Control U
<code>c_cc[VEOF]</code>	EOF(파일의 끝)	Control D
<code>c_cc[VEOL]</code>	EOL(추가된 라인의 끝)	NULL
<code>c_cc[VSUSP]</code>	SUSP	Control Z
<code>c_cc[VSTART]</code>	START	Control Q
<code>c_cc[VSTOP]</code>	STOP	Control S

- 각각의 터미널 모드와 관련 제어 문자들을 자세히 알고 있어야 터미널 장치를 잘 제어할 수 있다. 자세한 내용은 터미널 제어에 대한 문서를 좀더 참조하도록 하고 여기에서는 자주 사용되는 예를 이용해보도록 하자.

터미널 장치의 속성 제어

- 일반적으로 로컬 모드에는 ICANON 모드가 설정되어 있어 입력 시 문자들을 버퍼링하면서 제어 문자들을 처리하다가 <Enter> 문자가 들어오면 입력 버퍼의 내용을 프로그램에 전달한다. 이 ICANON 모드를 해제하면 RAW 모드로 설정되고, 이 모드에서는 제어 문자가 의미가 없으므로 `c_cc[VEOF]`에 있는 EOF는 아무 의미 없는 문자가 되며, 그 대신 MIN과 TIME의 의미로 사용된다. MIN과 TIME은 입력 문자의 길이가 최소 MIN에 해당하고 1/10초 TIME 배의 시간이 경과하면 입력 문자가 프로그램에 전달되도록 하는 기준 값이다.
- 이 값은 ICANON 모드에서는 의미가 없고 오로지 RAW 모드에서만 유효하다.
- RAW 모드에서의 `c_cc` 배열의 제어 문자는 다음과 같은 의미를 지닌다.

배열 원소	의미
<code>c_cc[VINTR]</code>	INTR 문자
<code>c_cc[VMIN]</code>	입력 시 최소 글자 수
<code>c_cc[VQUIT]</code>	QUIT 문자
<code>c_cc[VSUSP]</code>	SUSP 문자
<code>c_cc[VTIME]</code>	입력 완료 시간
<code>c_cc[VSTART]</code>	START 문자
<code>c_cc[VSTOP]</code>	STOP 문자

터미널 장치의 속성 제어

- 이제 `ioctl()` 함수를 이용하여 터미널 장치의 속성을 제어해보자.
- 프로그램을 작성하다 보면 사용자가 키보드에서 키를 누르자마자 입력을 받아 처리해야 하는 경우가 있다. 예를 들면 로그인시 비밀번호 입력 과정에 키누르는 것이 보이지 않도록 설정하면서 대신 키를 누를 때마다 ‘*’ 문자가 나타나게 해주기로 하는 경우다. 이런 경우 터미널 장치의 속성을 RAW 모드로 설정하고 `getchar()` 또는 `fgetc()` 함수를 이용하게 되면 버퍼링 없이 입력 받게 된다.

〈참고〉

리눅스에서 터미널을 제어할 때에는 일반적으로 다음과 같은 두 가지 함수를 이용한다.

```
tcgetattr(int fd, struct termios *tbuf);
```

```
tcsetattr(int fd, int optional_actions, const struct termios *tbuf);
```

이 두 함수는 `ioctl()`을 통해 구현된 매크로 함수로 터미널 전용의 함수이다.

소스코드 (ex02.c)

```
.....
#include <termio.h>
#define CR '\012'
int main(int argc, char *argv[])
{
    struct termio tbuf, oldtbuf;
    char ch;
    if(ioctl(0, TCGETA, &tbuf) == -1) { // 현재 터미널 모드
        perror("ioctl");
        exit(1);
    }
    oldtbuf = tbuf;
    tbuf.c_lflag &= ~ICANON; // ICANON 모드 해제 --> RAW 모드
    tbuf.c_cc[VMIN] = 1; // 입력 문자 최소 수
    tbuf.c_cc[VTIME] = 0; // 입력 완료 시간
    if(ioctl(0, TCSETAF, &tbuf) == -1) {
        perror("ioctl");
        exit(1);
    }
    while(1) {
        ch = getchar();
        if(ch == CR)
            break;
        printf("%x", ch);
    }
    if(ioctl(0, TCSETAF, &oldtbuf) == -1) { // 원래 모드로 재 설정
        perror("ioctl");
        exit(1);
    }
    return 0;
}
```

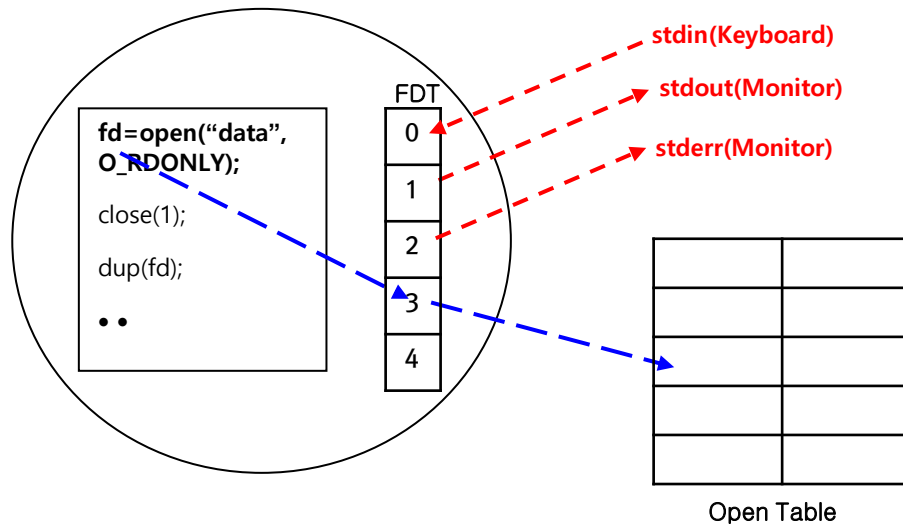
파일 중첩

- 리눅스에서 파일은 파일 디스크립터를 이용하여 제어한다. 이 파일 디스크립터를 통해 데이터 입출력도 할 수 있지만 서로 중첩시켜 서로 다른 파일 디스크립터가 동일한 파일을 접근할 수 있게 할 수도 있다. `dup()`, `dup2()` 함수가 이런 기능을 가지고 있는 함수로 이 함수는 파이프를 이용한 프로세스간의 통신이나 소켓 통신에도 적용하여 유용하게 사용할 수 있다. 이 함수의 원형은 다음과 같다.

```
#include <unistd.h>
```

```
int dup(int fd);  
int dup2 (int tofd, int fromfd);
```

- `dup()` 시스템 콜을 호출하면 우선 파일 디스크립터 테이블에서 유용한 파일 디스크립터 중 가장 낮은 번호의 파일 디스크립터를 찾아낸다. 그런 후 인수로 전달한 파일 디스크립터에 설정된 정보를 앞서 찾아낸 디스크립터로 복사해준다. 즉, 다음 그림과 같다.



파일 중첩

- `dup2()` 함수도 `dup()` 함수와 동일한 기능을 수행하나 인수가 2개 전달된다. 첫번째 인수는 복사할 파일 디스크립터를, 두 번째 인수에는 기존의 접근 정보를 종료하고 새로운 정보로 복사될 파일 디스크립터를 전달한다. 두 파일 디스크립터가 동일하면 `dup2()` 함수는 아무 일도 수행하지 않는다. 위의 그림 예에서 사용하였던

```
close(1);  
dup(fd);
```

를 `dup2()` 함수로 표현하면 다음과 같다.

```
dup2(fd, 1);
```

- 이 함수는 모든 파일 디스크립터에 적용이 가능하다. 즉, 파일 스크립터를 이용하는 모든 파일에 유용하게 사용할 수 있다.

ex03.c 를 실행하여 결과를 확인하고,
dup() 함수를 이용하여 write()와 printf() 함수의 수행결과가 duptest 파일에 저장되도록 빈 칸을 추가해보자

소스코드 (ex03.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main(void)
{
    int fd;
    char buffer[]="Hello, dup!!\n";

    if((fd=open("duptest",O_WRONLY|O_CREAT|O_TRUNC,0666))===-1){
        perror("open");
        exit(1);
    }

    write(1, buffer, strlen(buffer));
    printf("This is dup() test file\n");
    return 0 ;
}
```

파일 중첩

- 앞서 살펴본 ex03.c는 1번 디스크립터를 다른 파일과 중첩시킨 예제 프로그램이었다. 0, 1, 2번 뿐만 아니라 어떤 파일 디스크립터도 다른 디스크립터와 중첩이 가능하다.

디렉토리 제어

1. 디렉토리 접근
2. 디렉토리 제어

디렉토리 접근

■ 디렉토리 구조

- 리눅스에서 디렉토리는 다음과 같은 구조로 구성되어 있는 파일이다.

```
struct dirent {  
    ino_t d_ino;           // inode 번호  
    off_t d_off;  
    unsigned short d_reclen;  
    char d_name[256]; // 파일명  
};
```

- 위의 자료형은 dirent.h 헤더 파일에 선언되어 있는 구조체로 디렉토리 파일은 구조체 자료형의 모임으로 구성되어 있다. 디렉토리에는 해당 디렉토리 아래에 있는 파일에 대한 inode 번호와 파일 명이 등록되어 있어 파일에 대한 정보를 찾아야 할 때 디렉토리 파일로부터 파일에 대한 inode 번호를 알아온다. 여기에서 알아낸 inode 번호를 이용하여 파일시스템의 inode 리스트로부터 정보를 찾아오게 된다.
- 따라서 디렉토리 파일로 접근하려면 일반 파일과 같은 open(), read(), write(), close() 함수를 이용할 수 없다. 디렉토리 파일만 전담하는 함수로 opendir(), closedir(), readdir() 함수를 이용해야 한다.

디렉토리 접근

- `opendir()`, `closedir()` 함수
- `opendir()` 함수는 디렉토리 파일에 접근이 가능하게 해주는 함수로 원형은 다음과 같다.

```
#include <sys/types.h>
#include <dirent.h>
```

```
DIR *opendir(const char *name);
int closedir(DIR * dirp);
```

- `opendir()` 함수를 성공하면 디렉토리 파일을 읽을 수 있는 `DIR *`의 포인터를 반환하고, 실패하면 `NULL`을 반환한다. `closedir()` 함수는 디렉토리 파일에 대한 접근을 해제할 때 사용하는 함수로 `opendir()` 함수의 반환 값을 인수로 전달한다.

디렉토리 접근

- readdir() 함수
- readdir() 함수를 이용하면 디렉토리 파일 안에 있는 엔트리를 읽어올 수 있다. 이 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <dirent.h>

struct dirent *readdir(DIR *dirp);
```

- readdir() 함수는 디렉토리 엔트리를 순차적으로 읽어 반환하고, 끝까지 다 읽으면 NULL을 반환한다. 엔트리로 읽어 온 각 파일에 대한 세부항목은 앞서 학습한 stat() 함수를 이용하여 알 수 있다



파일명을 전달 받아 파일의 유형을 확인하고 디렉토리인 경우 디렉토리 목록을 출력하는 ex04.c 를 작성해봅시다

디렉토리 제어

- mkdir(), rmdir() 함수

- 이 시스템 콜을 이용하면 디렉토리를 생성하거나 삭제할 수 있다. 함수의 원형은 다음과 같다.

```
#include <sys/stat.h>
int mkdir(const char *path, mode_t mode);
```

```
#include <unistd.h>
int rmdir(const char *path.);
```

- mkdir() 함수는 path에 디렉토리 명을, mode에는 디렉토리 권한을 지정하여 호출하면 디렉토리를 생성한다. rmdir() 함수는 path에 삭제할 디렉토리 명을 전달하면 디렉토리를 삭제한다. 이때 해당 디렉토리는 비어있어야 한다. 만약 디렉토리 파일이 비어있지 않아도 삭제를 원하는 경우에는 4장에서 파일 삭제 시 이용하는 unlink() 함수를 이용한다.

디렉토리 제어

■ chdir(), getcwd() 함수

- 프로그램을 수행하다가 다른 디렉토리로 이동하는 경우에는 chdir() 함수를 이용한다. 그리고 현재 작업 디렉토리의 경로를 확인하려면 getcwd() 함수를 이용한다. 각 함수의 원형은 다음과 같다.

```
#include <unistd.h>
```

```
int chdir(const char *path);
```

```
char * getcwd(char *buf, size_t size);
```

- chdir() 함수에는 변경하고자 하는 경로 명을 인수로 전달한다. 디렉토리 이동이 성공하면 0을, 실패하면 -1을 반환한다. 이때 오류의 원인은 errno 변수에 설정된다.(man 참조)
- getcwd() 함수는 현재의 경로 위치를 첫번째 인수 buf에 넘겨준다. 이때 두 번째 인수size로 전달하는 크기보다 경로명이 긴 경우에는 NULL 을 반환한다. 오류의 원인은 errno 변수에 설정된다. (man 참조)

다음과 같은 ex05.c 를 실행하며 디렉토리 제어 함수를 이해해보자.

```
...  
  
int main(void)  
{  
    char cmdbuf[256];  
    char pathbuf[1024];  
  
    mkdir("testdir", 0755);  
    strcpy(cmdbuf, "ls -l | grep ");  
    strcat(cmdbuf, "testdir");  
    printf("%s\n", cmdbuf);  
    system(cmdbuf);  
  
    chdir("testdir");  
    if(getcwd(pathbuf, 1023) == NULL) {  
        perror("getwd");  
        exit(1);  
    }  
    printf("Current Directory : %s\n", pathbuf);  
  
    chdir("..");    // 상위 디렉토리로 이동  
    rmdir("testdir");  
    printf("%s\n", cmdbuf);  
    system(cmdbuf);  
    return 0;  
}
```

〈실습〉

- 다음 프로그램은 표준 입력 장치인 키보드로부터 입력 받아 출력하는 프로그램이다. 이번 시간에 학습한 내용 중 파일 중첩 기능을 이용하여 다음 프로그램이 파일로부터 입력이 가능하도록 완성하시오.

소스코드 (exercise.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    char buffer[80];

    printf("Please input a string --> ");
    fgets(buffer, 80, stdin);
    printf("You input the sting : %s\n", buffer);
    return 0;
}
```